

Práctica 1

Un Estudiante de Sistemas Operativos

5 de octubre de 2025

Índice general

1. Ejercicio 1	1
1.1. Enunciado del Ejercicio	1
1.1.1. Estructura del Árbol de Procesos	1
1.1.2. Resultado Esperado	1
1.2. Identificación y Justificación de Tareas	3
1.2.1. Recepción y Validación de Argumentos	3
1.2.2. Generación de la Jerarquía de Procesos	3
1.2.3. Temporización sin Bloqueo	3
1.2.4. Ejecución de la Orden Mediante Señales	3
1.2.5. Comportamiento Reactivo y Ejecución de Comandos	4
1.2.6. Notificación y Transición a la Fase de Destrucción	4
1.2.7. Destrucción Controlada del Árbol	4
1.3. Diagrama de Secuencia	5
1.4. Análisis Detallado del Código Fuente	6
1.4.1. Funciones Globales y de Utilidad	7
Función <code>parse_args()</code>	7
1.4.2. Proceso <code>ejec</code> (El Súper-padre)	7
Función <code>main()</code>	7
Llamadas al Sistema Utilizadas:	7
Función <code>handler_start_destruction()</code>	8
Llamadas al Sistema Utilizadas:	8
1.4.3. Proceso A	8
Función <code>create_A_process()</code>	8
Llamadas al Sistema Utilizadas:	8
Función <code>run_process_A()</code>	8
Llamadas al Sistema Utilizadas:	8
Función <code>handler_A_exec_task()</code>	9
Llamadas al Sistema Utilizadas:	9
Función <code>handler_A_destroy_and_propagate()</code>	9
Llamadas al Sistema Utilizadas:	9
1.4.4. Proceso B	9
Función <code>create_B_process()</code>	9
Llamadas al Sistema Utilizadas:	9
Función <code>run_process_B()</code>	10
Llamadas al Sistema Utilizadas:	10
Función <code>handler_B_exec_task()</code>	10
Llamadas al Sistema Utilizadas:	10
Función <code>handler_B_destroy_and_propagate()</code>	10

	Llamadas al Sistema Utilizadas:	11
1.4.5.	Proceso X	11
	Función <code>create_X_process()</code>	11
	Llamadas al Sistema Utilizadas:	11
	Función <code>run_process_X()</code>	11
	Llamadas al Sistema Utilizadas:	11
	Función <code>handler_X_exec_task()</code>	11
	Llamadas al Sistema Utilizadas:	12
	Función <code>handler_X_destroy_leaf()</code>	12
	Llamadas al Sistema Utilizadas:	12
1.4.6.	Proceso Y	12
	Función <code>create_Y_process()</code>	12
	Llamadas al Sistema Utilizadas:	12
	Función <code>run_process_Y()</code>	12
	Llamadas al Sistema Utilizadas:	13
	Función <code>handler_Y_exec_task()</code>	13
	Llamadas al Sistema Utilizadas:	13
	Función <code>handler_Y_destroy_leaf()</code>	13
	Llamadas al Sistema Utilizadas:	13
1.4.7.	Proceso Z	13
	Función <code>create_Z_process()</code>	13
	Llamadas al Sistema Utilizadas:	14
	Función <code>run_process_Z()</code>	14
	Llamadas al Sistema Utilizadas:	14
	Función <code>handler_Z_alarm()</code>	14
	Llamadas al Sistema Utilizadas:	14
	Función <code>handler_Z_destroy_leaf()</code>	15
	Llamadas al Sistema Utilizadas:	15

Capítulo 1

Ejercicio 1

1.1. Enunciado del Ejercicio

Realiza un programa llamado `ejec.c` que reciba dos argumentos. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el segundo argumento, le ordenará mediante una señal al proceso identificado con el primer argumento (A, B, X, Y) que ejecute una orden. Si la señal la recibe el proceso A o B ejecutarán el comando “`pstree`”. Si la señal la recibe el proceso X o Y ejecutarán el comando “`ls`”. El proceso Z no puede utilizar la llamada al sistema “`sleep`” para realizar la espera.

Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).

1.1.1. Estructura del Árbol de Procesos

La jerarquía de procesos que se debe crear es la siguiente:

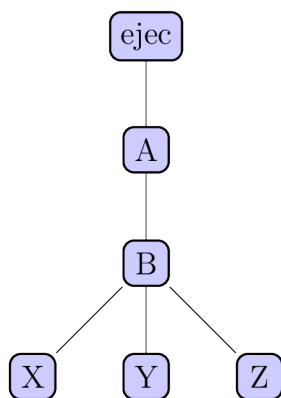


Figura 1.1: Diagrama del árbol de procesos a generar.

1.1.2. Resultado Esperado

A continuación se muestra un ejemplo de la salida esperada por la terminal:

```
$ ejec A 15
Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
```

```
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752...
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752...

/* Tras un intervalo de 15 segundos aparecera */
Soy el proceso A con pid 752, he recibido la senyal.
/* Resultado del comando pstree */

Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero
```

1.2. Identificación y Justificación de Tareas

La arquitectura del programa se deriva directamente de un análisis detallado de los requisitos del enunciado. Cada frase, ya sea explícita o implícita, se traduce en una decisión de diseño y en un conjunto de tareas de programación específicas que se describen a continuación.

1.2.1. Recepción y Validación de Argumentos

El enunciado comienza con la frase: *“Realiza un programa llamado `ejec.c` que reciba dos argumentos.”* Esto establece el requisito fundamental de que el programa debe interactuar con el usuario a través de la línea de comandos. La justificación de esta tarea es la necesidad de configurar el comportamiento del programa en tiempo de ejecución, específicamente para determinar qué proceso será el objetivo de la señal y cuánto tiempo debe esperar el temporizador. Para llevar a cabo esta tarea, la función `main()` delega la responsabilidad de la lectura y validación a la función `parse_args()`. Esta última se especializa en verificar que el número de argumentos sea el correcto, convertirlos a los tipos de datos necesarios y almacenarlos en variables globales.

1.2.2. Generación de la Jerarquía de Procesos

La restricción *“El programa tendrá que generar el árbol de procesos que se indica”* es un pilar del diseño. Esto requiere una serie de creaciones de procesos anidadas y controladas para replicar la estructura `ejec ->A ->B ->{X,Y,Z}`. Para lograr una implementación clara y modular, esta tarea se descompone en un conjunto de funciones, donde cada una es responsable de una única bifurcación. Las funciones `create_A_process()`, `create_B_process()`, `create_X_process()`, etc., encapsulan una llamada a `fork()` y la lógica asociada, permitiendo construir el árbol paso a paso.

1.2.3. Temporización sin Bloqueo

El enunciado impone dos condiciones clave para la temporización: *“El proceso Z, transcurridos los segundos indicados...”* y *“...no puede utilizar la llamada al sistema ‘sleep’.”* Esto obliga a implementar un mecanismo de espera asíncrono. La justificación es evitar un bloqueo ineficiente del proceso Z. El mecanismo estándar en UNIX para este fin es el uso de alarmas. La implementación recae en la función `run_process_Z()`, que utiliza la llamada al sistema `alarm()` para solicitar al sistema operativo que le envíe una señal `SIGALRM` en el futuro. Inmediatamente después, el proceso se suspende de forma eficiente con `pause()`, sin consumir CPU hasta que la señal de la alarma lo despierte.

1.2.4. Ejecución de la Orden Mediante Señales

El evento central se describe como: *“...le ordenará mediante una señal al proceso identificado con el primer argumento...”*. Esto define el rol del proceso Z como el iniciador de la acción principal. Una vez que Z es “despertado” por su alarma, se ejecuta su manejador, `handler_Z_alarm()`. La tarea de este manejador es identificar al proceso objetivo a partir de la configuración global y enviarle la señal de ejecución. Para ello, utiliza la llamada al sistema `kill()`, enviando la señal `SIGUSR1` al PID del proceso correcto.

1.2.5. Comportamiento Reactivo y Ejecución de Comandos

La reacción de los procesos objetivo se especifica con: *“Si la señal la recibe... ejecutarán el comando...”*. Los procesos A, B, X e Y deben tener un comportamiento reactivo, definido en sus respectivos manejadores `handler*_exec_task()`. Para cumplir con el requisito de “ejecutar un comando” sin que el proceso principal termine, la ejecución debe delegarse en un proceso hijo temporal. Dentro de cada manejador, se utiliza `fork()` para crear este hijo efímero. Es este nuevo proceso el que invoca a `execlp()` para reemplazarse a sí mismo por el comando especificado (`ps` o `ls`), dejando intacto al proceso original.

1.2.6. Notificación y Transición a la Fase de Destrucción

Aunque no se menciona explícitamente, existe un requisito implícito de que el sistema debe saber cuándo iniciar la fase de apagado. Lógicamente, este evento debe ser desencadenado por el proceso que completó la tarea. La implementación de esta transición ocurre dentro de los manejadores `handler*_exec_task()`. Después de que el hijo temporal que ejecutó el comando haya terminado, el padre original es desbloqueado de su llamada a `wait()`. En este momento, el manejador cumple su segunda responsabilidad: notificar a la raíz del sistema (`ejec`) que la misión ha sido cumplida. Para ello, utiliza `kill()` para enviar una segunda señal, `SIGUSR2`, al proceso `ejec`.

1.2.7. Destrucción Controlada del Árbol

Finalmente, la restricción más importante sobre la terminación es: *“Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos).”* Esto impone una cascada de apagado controlada. La señal `SIGUSR2` actúa como el interruptor de esta fase. Al recibirla, la función `handler_start_destruction()` en `ejec` inicia la cascada enviando la misma señal a su hijo A. A partir de ahí, cada padre en la cadena, a través de su manejador `handler*_destroy_and_propagate()`, tiene la doble tarea de propagar la señal a sus propios hijos y, fundamentalmente, esperar a que estos terminen (usando `wait()`) antes de finalizar él mismo. Los procesos hoja (X, Y, Z) simplemente terminan al recibir la señal en sus manejadores `handler*_destroy_leaf()`, desbloqueando a sus padres y permitiendo que la secuencia de apagado ascienda correctamente por el árbol.

1.3. Diagrama de Secuencia

El siguiente diagrama ilustra el ciclo de vida completo, mostrando el anidamiento de las llamadas a funciones, la interrupción de las activaciones durante los estados de espera y la cascada de destrucción controlada.

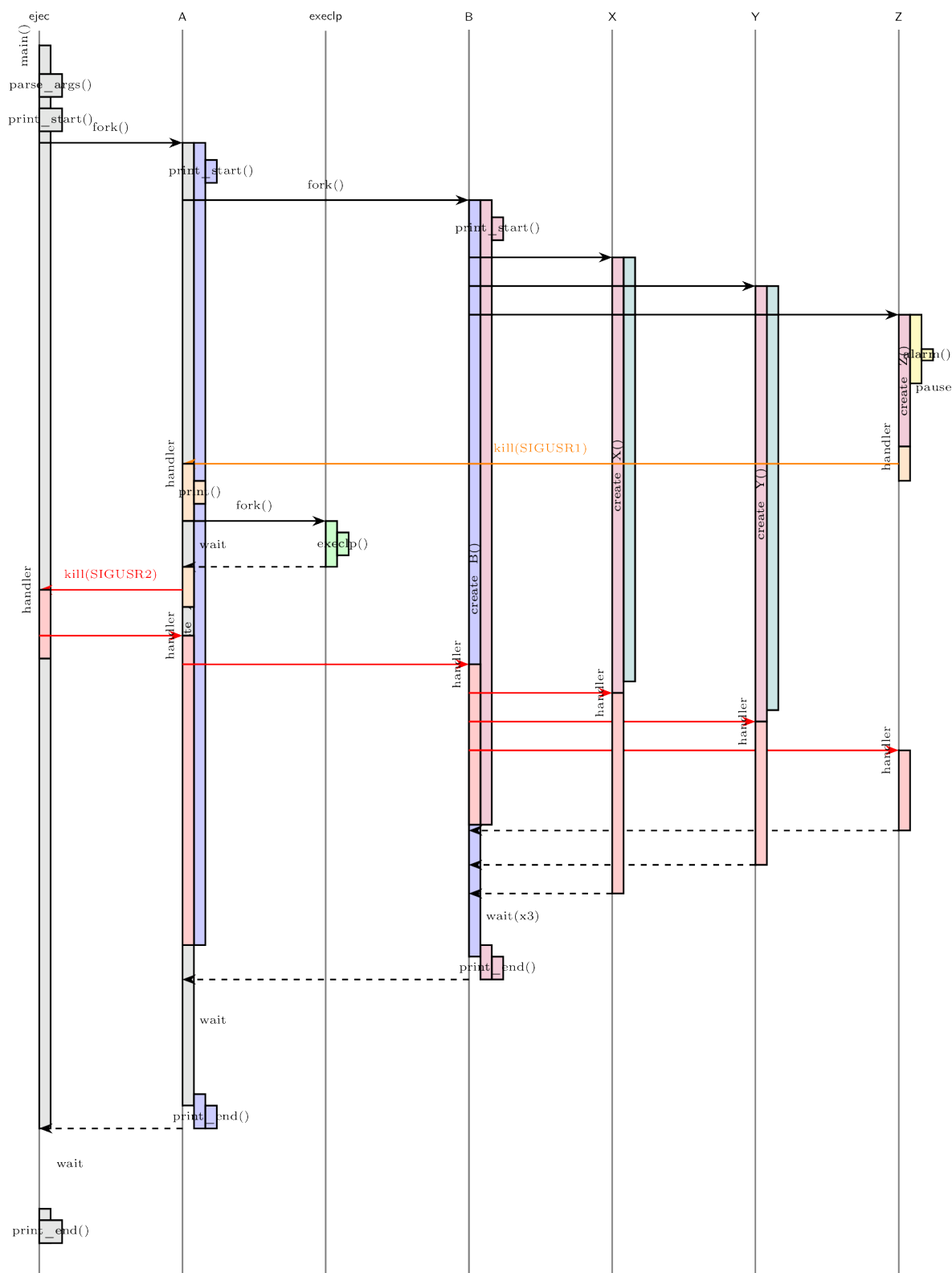


Figura 1.2: Diagrama de secuencia definitivo con anidamiento estricto, etiquetas completas y cajas de activación partidas.

1.4. Análisis Detallado del Código Fuente

A continuación, se desglosa el propósito y funcionamiento de cada una de las funciones definidas en el programa `ejec.c`, segmentando el análisis por cada uno de los procesos

que componen el árbol.

1.4.1. Funciones Globales y de Utilidad

Función parse_args()

El propósito de esta función es actuar como la única puerta de entrada para los parámetros externos del programa. Su responsabilidad es interpretar los argumentos de la línea de comandos, validarlos para asegurar que el programa se ha invocado correctamente, y configurar el estado inicial de la aplicación almacenando dichos parámetros en variables globales.

```

1 void parse_args(int argc, char *argv[]) {
2     if (argc != 3) {
3         fprintf(stderr, "Uso: %s <proceso> <segundos>\n", argv[0]);
4         exit(EXIT_FAILURE);
5     }
6     g_target_process_char = argv[1][0];
7     g_seconds = atoi(argv[2]);
8
9     if (g_target_process_char != 'A' && g_target_process_char != 'B' &&
10        g_target_process_char != 'X' && g_target_process_char != 'Y') {
11         fprintf(stderr, "Proceso objetivo invalido.\n");
12         exit(EXIT_FAILURE);
13     }
14 }
```

1.4.2. Proceso ejec (El Súper-padre)

Función main()

Esta función es el punto de entrada principal y actúa como el orquestador general del ciclo de vida de la aplicación. Su misión es secuenciar las tres fases principales: inicialización (procesando argumentos e identificándose), ejecución (lanzando la creación del árbol de procesos y esperando su completa resolución) y finalización (imprimiendo su mensaje de muerte). Una tarea clave que realiza es registrar el manejador que escuchará el evento de "tarea completada" para poder iniciar la destrucción del sistema.

Llamadas al Sistema Utilizadas: getpid(), signal(), wait().

```

1 int main(int argc, char *argv[]) {
2     parse_args(argc, argv);
3     g_pid_ec = getpid();
4     printf("Soy el proceso ejec: mi pid es %d\n", g_pid_ejec);
5
6     signal(SIGUSR2, handler_start_destruction);
7     create_A_process();
8
9     printf("Soy ejec (%d) y muero\n", g_pid_ejec);
10    return 0;
11 }
```

Función handler_start_destruction()

El objetivo de este manejador es actuar como el punto de inicio de la cascada de terminación controlada. Se activa exclusivamente en el proceso `ejec` cuando recibe la señal que notifica el fin de la tarea. Su única responsabilidad es propagar la orden de destrucción al primer eslabón de la cadena jerárquica, el proceso A.

Llamadas al Sistema Utilizadas: `kill()`.

```
1 void handler_start_destruction(int s) {
2     kill(g_pid_a, SIGUSR2);
3 }
```

1.4.3. Proceso A**Función create_A_process()**

El propósito de esta función es crear el proceso A y, crucialmente, hacer que su padre (`ejec`) espere a que toda la rama de descendientes de A complete su ciclo de vida. Esto garantiza la restricción de que ningún padre muera antes que sus hijos.

Llamadas al Sistema Utilizadas: `fork()`, `wait()`.

```
1 void create_A_process() {
2     switch (g_pid_a = fork()) {
3         case 0:
4             run_process_A();
5             exit(0);
6         default:
7             wait(NULL);
8     }
9 }
```

Función run_process_A()

El objetivo de esta función es definir la fase de inicialización y vida del proceso A. Sus tareas son: registrar su identidad (PID), imprimir su mensaje de nacimiento, y establecer su comportamiento reactivo registrando los manejadores para las señales de ejecución y destrucción. Finalmente, cumple su rol estructural continuando la construcción del árbol al invocar la creación de su hijo, el proceso B.

Llamadas al Sistema Utilizadas: `getpid()`, `signal()`.

```
1 void run_process_A() {
2     g_pid_a = getpid();
3     printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n",
4           g_pid_a, g_pid_ejec);
5     signal(SIGUSR1, handler_A_exec_task);
6     signal(SIGUSR2, handler_A_destroy_and_propagate);
7     create_B_process();
8 }
```